

# Progressive (lazy) startup — implementation status

Branch: `feat/lightweight-progressive-startup` . Target: **v2.0.8**. Plan: `~/ .claude/plans/gentle-jingling-matsumoto.md` .

⚠ **NONE of this is end-to-end tested.** There is no Mac-side docker and no running VM in this session, so everything below is **static-validated only** ( `bash -n` , `py_compile` , YAML parse). **The fresh-VM launch test in "Verification" is mandatory before any release.** Do not cut a release yet.

## Done (static-validated)

### 1. Compose — Wave-1 lightweight default ( `docker-compose.appliance.yml` , YAML parses):

- `simulator now image: ${VYOMI_SIMULATOR_IMAGE:-${CLOUDLEARN_SIMULATOR_IMAGE:-vyomi/appliance:latest}}` + `build: . fallback` → fresh installs **pull the prebuilt ~400 MB image instead of building** (the launcher does `compose pull` first).
- Removed `simulator's depends_on` on the 10 backend containers → it starts without them.
- No profiles used (chose named-service waves — `simpler` + `back-compat`: a plain `docker compose up -d` still brings the full stack).

### 2. Launcher — Wave 1 / Wave 2 split ( `scripts/cloud-learn` , `bash -n` OK):

- `appliance_exec_launcher: Wave 1 = docker compose pull simulator caddy || true` then `up -d simulator caddy` (fast). Wave 2 = `systemd-run --unit=vyomi-wave2 ... docker compose up -d --remove-orphans` (detached background; `nohup` fallback) → streams the 10 backends + `cloudsim`.
- `appliance_health_check: READY now fires on bridge + simulator only` (dropped the `cloudsim :9010` requirement — it's Wave 2). `cloudsim` no longer reported as a launch failure.

### 3. Readiness gate mechanism:

- `core/appliance_readiness.py` : added `probe_all_cached(ttl=5)` + `is_ready()` (fail-OPEN on probe error). Reuses the existing weighted `probe_all()` + `GET /api/runtime/readiness` .
- `server.py` : added `_require_appliance_ready(action)` next to `_disk_preflight` — raises `503 {code:"appliance_not_ready", ready_pct, pending}` ; bypass `CLOUDLEARN_READINESS_GATE=disabled` ; auto-off outside appliance mode; fail-open.
- **Wired into `api_rds_create_database`** ( `server.py` ) as the proven representative.

## NOT done yet (scoped TODO)

- **Wire the gate into the rest of the create endpoints.** Only RDS is wired. Remaining Wave-2-dependent creates to gate (all need `import server; server._require_appliance_ready(...)` since they live in

provider modules): GCP Cloud SQL ( `providers/gcp_storage_sql_vpc.py:224` ), Azure SQL ( `core/azure_dataplane.py` create path / ARM), S3 ( `routes/aws_s3.py:1244` ), GCS ( `providers/gcp_services.py:152` ), DynamoDB ( `providers/aws_services.py:1327` ), SQS ( `providers/aws_services.py:1191` ), Pub/Sub ( `providers/gcp_services.py:538` ), Firestore ( `providers/gcp_services.py:1010` ).

- **Frontend (not started):** make the existing `clouds.html` readiness banner ( `pollReadiness` , lines ~1604-1652) **global** — extract to a shared snippet included by aws/gcp/azure consoles + dashboard, so the "Appliance is getting ready — N%" bar shows on **all views**. Disable Create/Launch buttons while `readiness.ready === false` . (Do the update-version banner in the same shared snippet — see the earlier "make banner global" request.)

## Open decisions (need your call)

---

1. **Gate compute VMs (EC2/GCE/Azure)?** They use **LXD**, which is ready after cloud-init — *before* Wave 2. Gating them on docker-backend readiness would block VMs that are actually ready (worse UX).  
**Recommendation: do NOT server-gate VMs** (only gate the docker-backed managed/data services); the UI can still disable their buttons during the "getting ready" window for a consistent look. Your "any service" ask vs. this semantic correctness — your call.
2. **Strictness of "ready":** `ready` currently means *all 12* backends up, incl. the ~1.4 GB GCP emulators + `cloudsim` (build). That can be several minutes. Consider per-service gating (allow S3 once minio is up) instead of global. Default left as global per your ask.

## Verification (MANDATORY before release)

---

1. Fresh `vyomi up` on a clean VM (test via `VYOMI_APPLIANCE_NAME=throwaway`): confirm `docker ps` shows **only simulator + caddy** first; `https://localhost:9443/` reachable + license-activatable in ~30–60 s; `systemctl status vyomi-wave2` shows Wave 2 running.
2. `watch GET /api/runtime/readiness` climbs as Wave 2 streams; reaches `ready:true` .
3. While not ready: `POST /api/rds/databases` → **503 appliance\_not\_ready** . After ready → succeeds.
4. `CLOUDLEARN_READINESS_GATE=disabled` → create allowed immediately.
5. Reuse path ( `vyomi up` on existing VM) still works.
6. Measure install time before/after (target: console usable in <1 min vs 5–10 min).